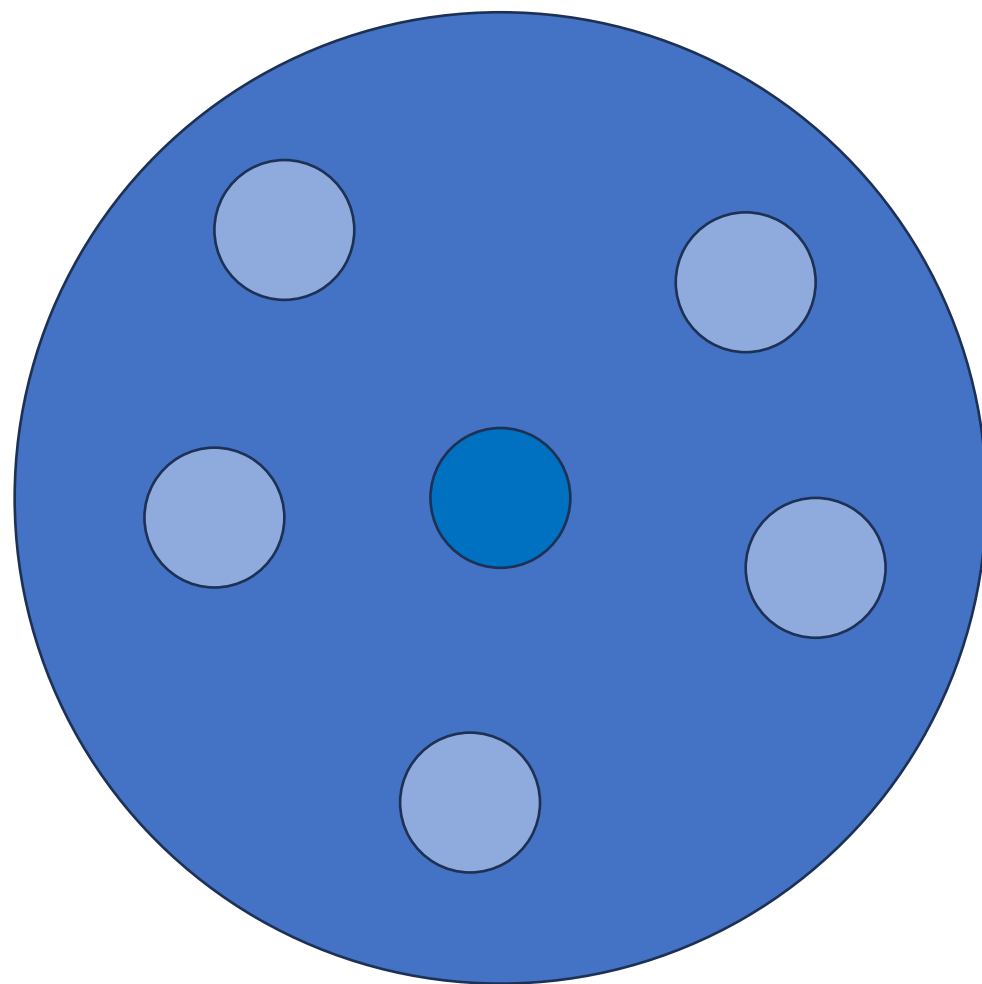


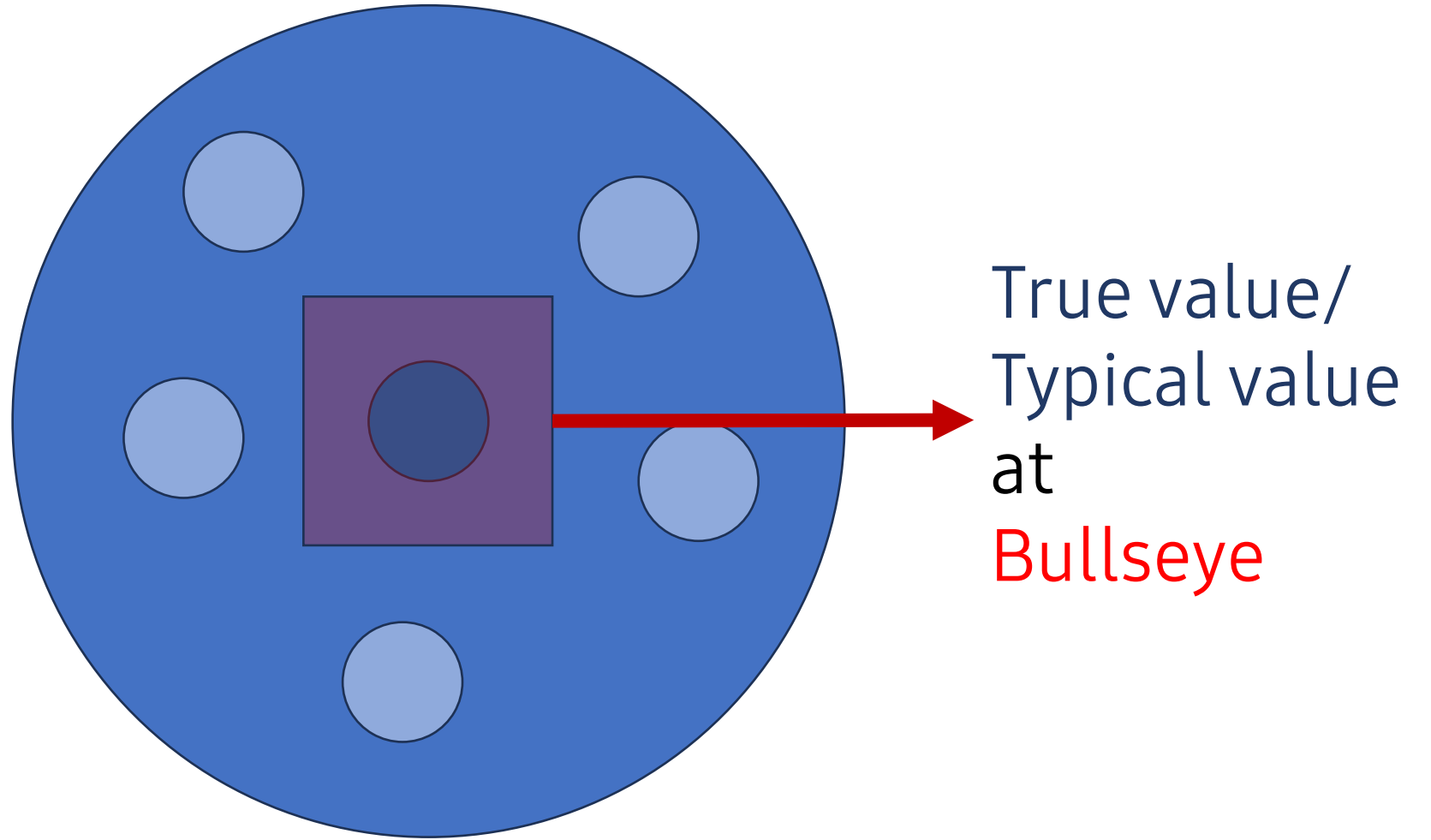
Day 2: Judgement and experiments

Lecturer: Msc. Minh Tan Le

Outline

- I. Judge the collections
- II. Experiments & simple simulation
- III. Python: Practices

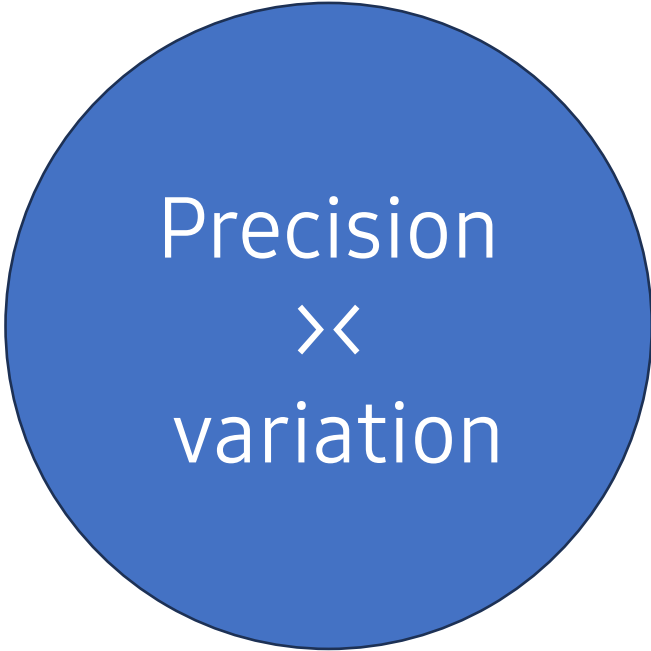




Accuracy



Bias

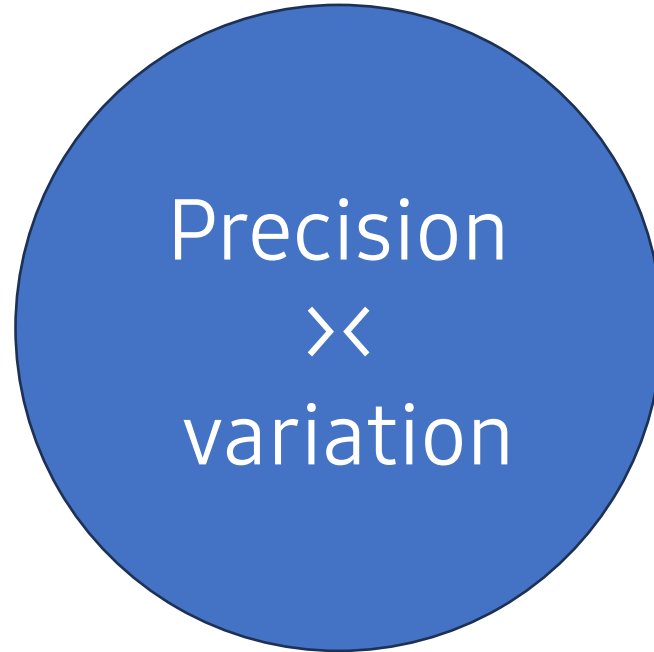


Precision
×
variation

Accuracy



Is the true value
centered?

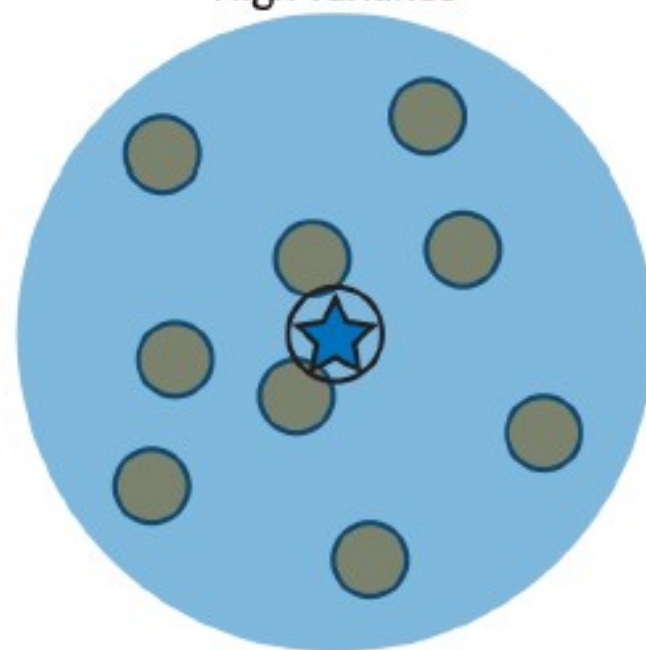
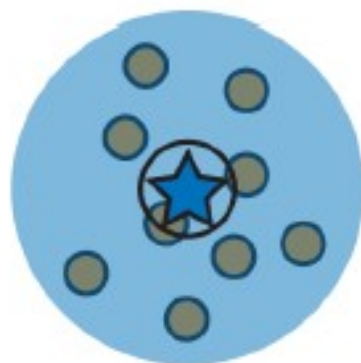


How far from the
bullseye?

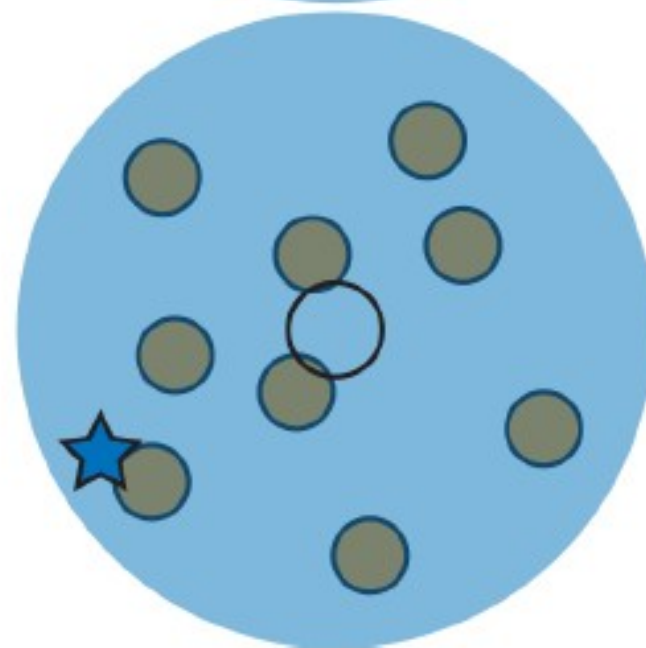
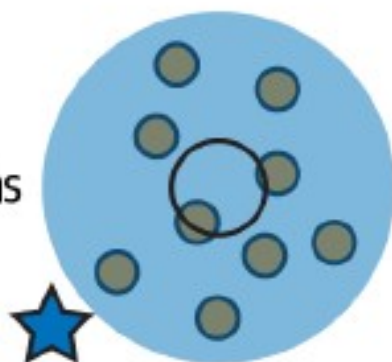
Low variance

High variance

Low bias



High bias



Types of bias

- Coverage bias: Access frame does not include specified target in the target population.
- Selection bias: Selection based on reach difficulty.
- Nonresponse bias: Target can be reached, but no response.
- Measurement bias: Technical problems (Measurement devices, environments,...)

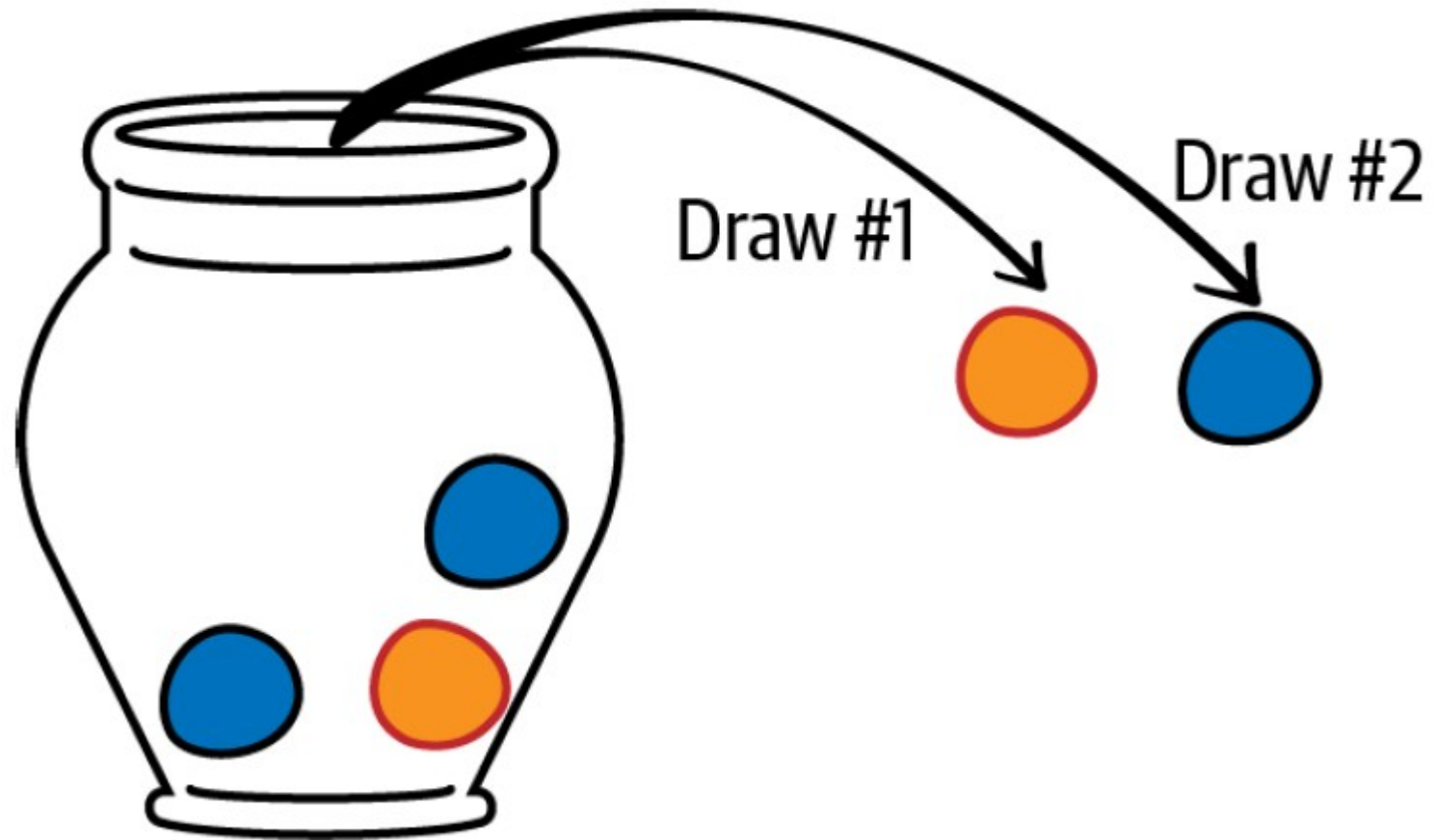
Types of variation

- Sampling variation: Affected by chance to select.
- Assignment variation: Controlled experiment affected by treatment assignments.
- Measurement error: Errors from measurement process.

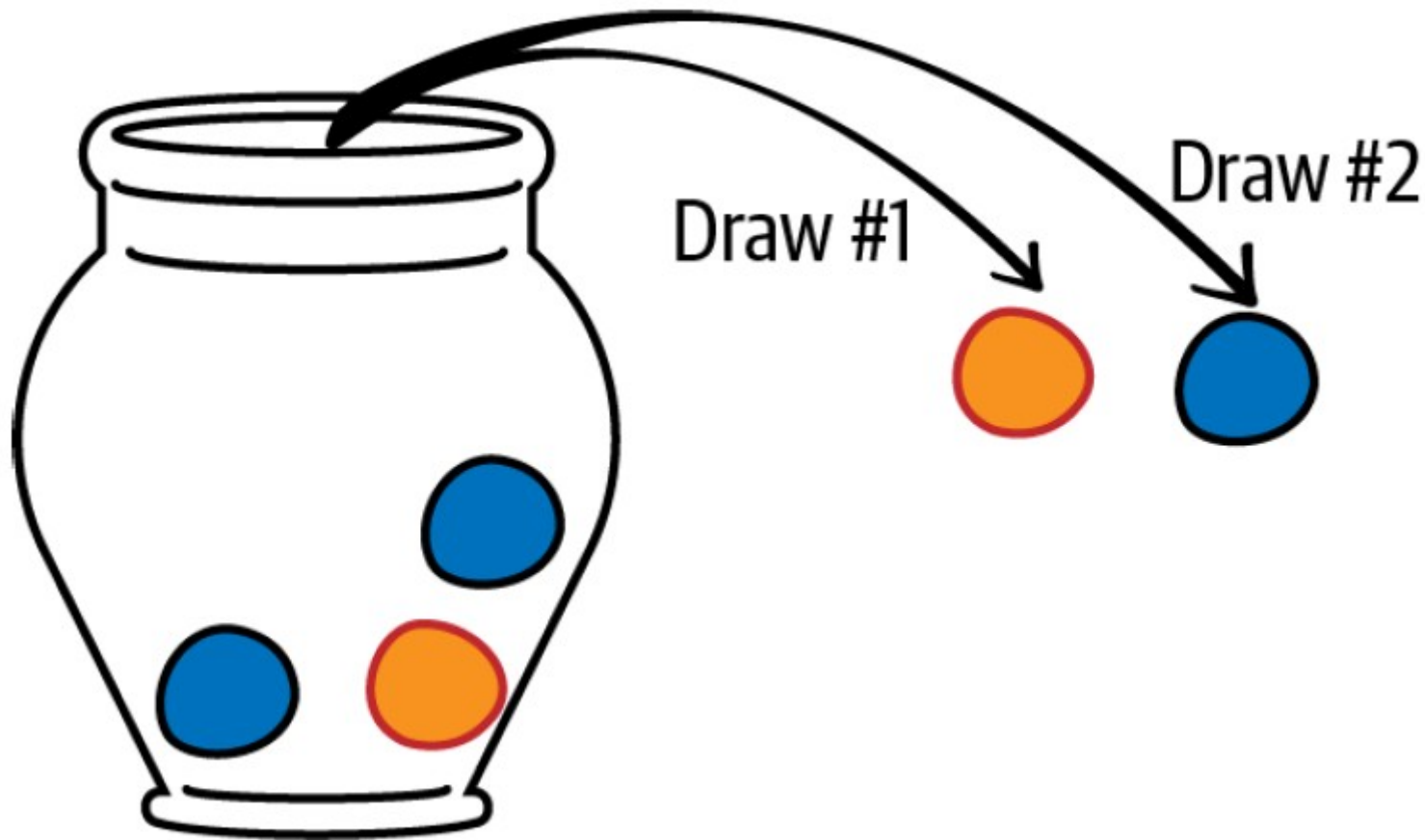
Summary

- Before diving deeper into preprocessing, exploration, analysis, if you didn't collect the data, ask yourself:
 - Who collected the data?
 - Why were the data collected?
 - When were the data collected?
 - Where were the data collected?

II. Simulation & experiment design



II. Simulation & experiment design

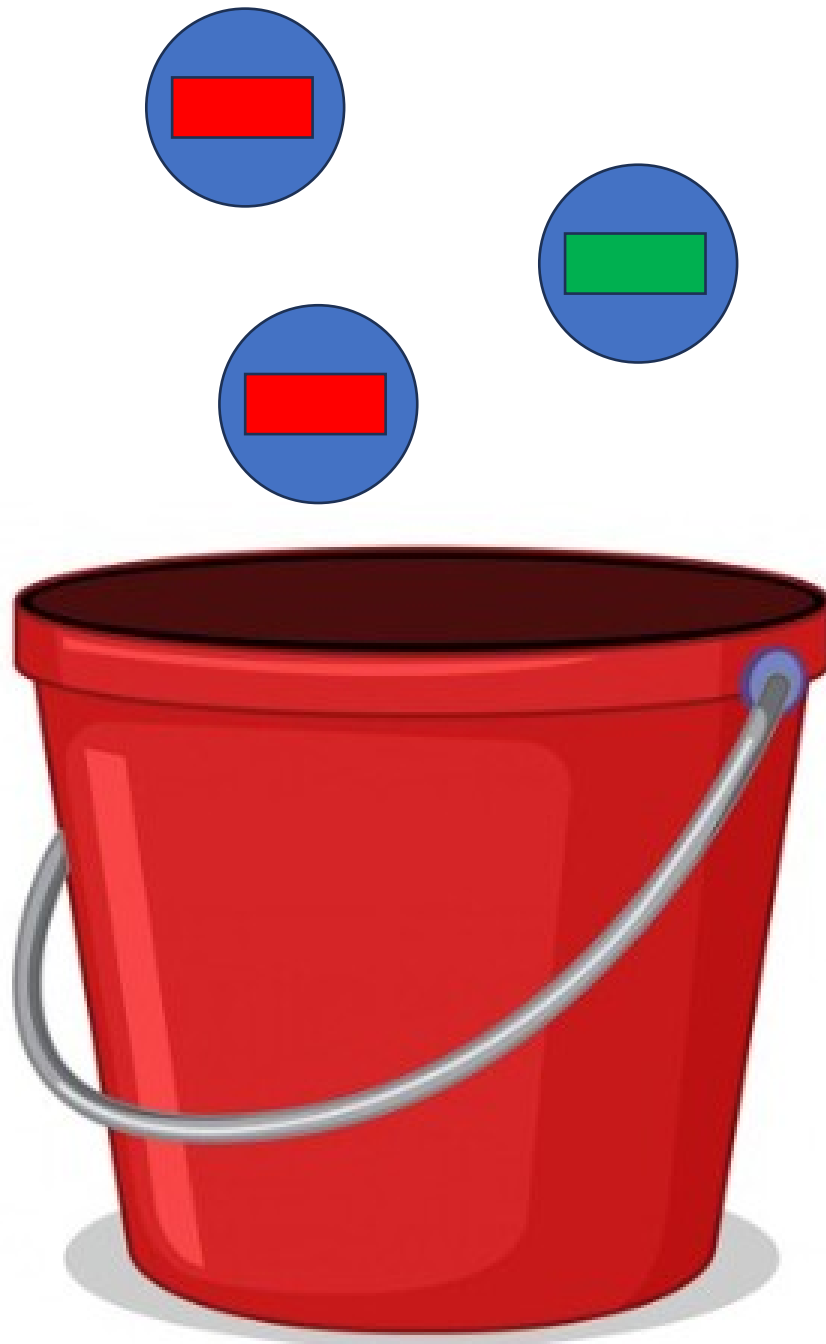


1. Set-up:

- Number of marbles
- The color on each marble
- The number of draws

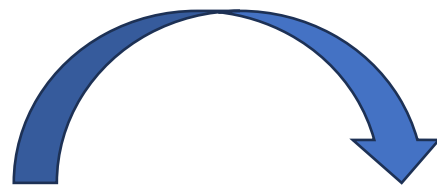


1m

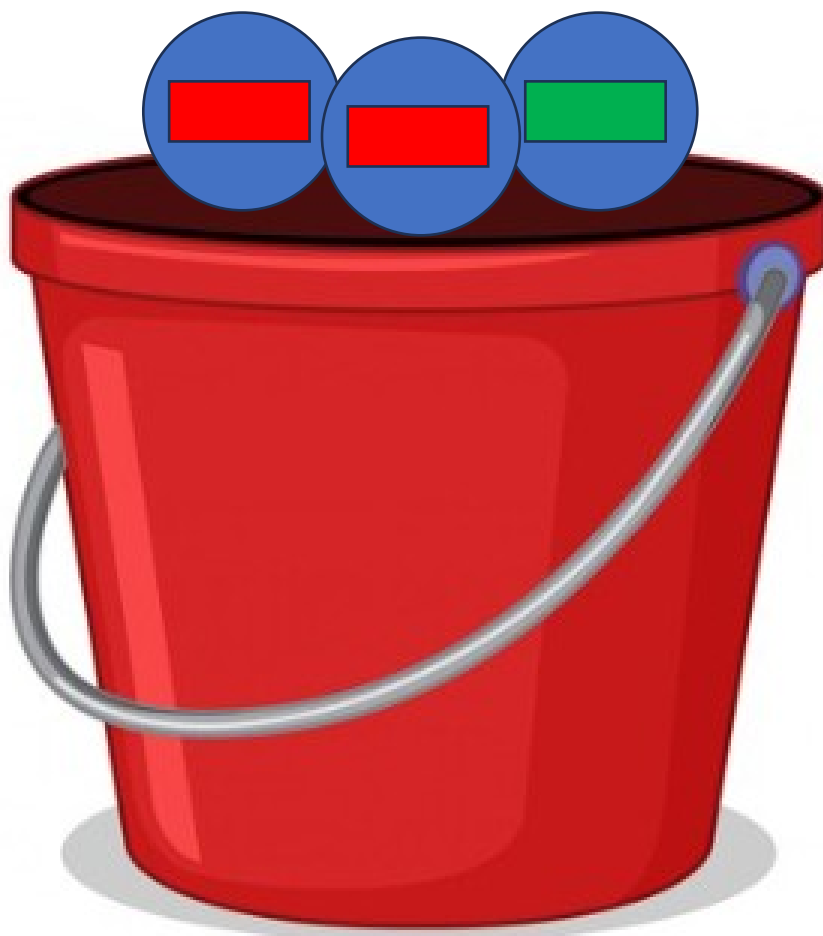


1m

Mixing

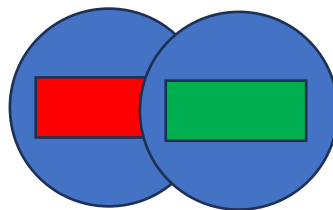
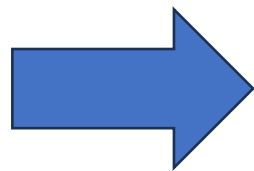
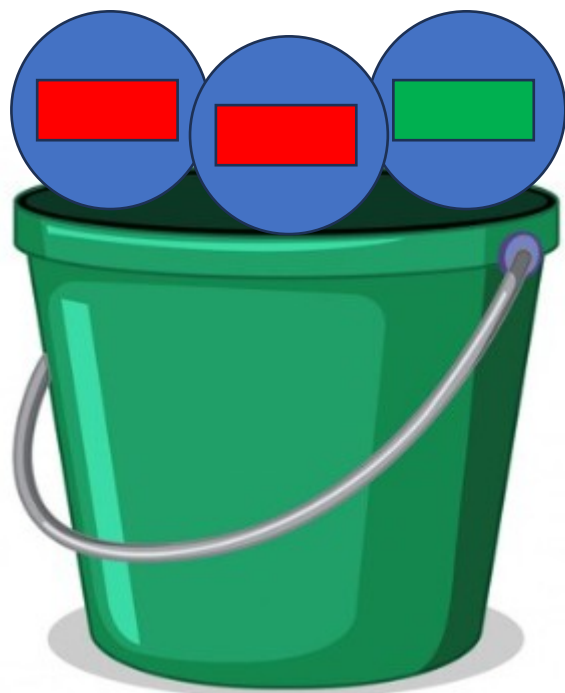


Picking one



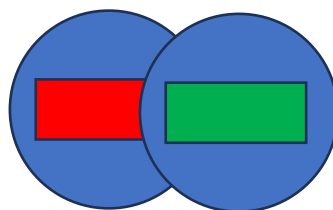
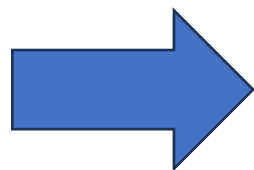
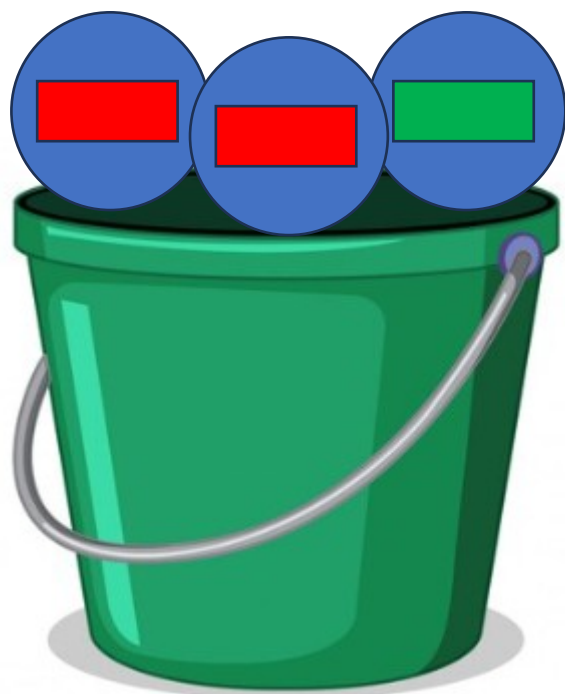
Total: 200

200



Total: 100

200



Total: 100

Urn exercise $\Rightarrow$ Urn model

What happen after picking a ball?

Record the color (label) and ...

With replacement

- Give **a same ball** back to the urn.
- Binomial distribution.

Without replacement

- Throw the ball away.
- Hypergeometric distribution.

What you're doing is **sampling design**

```
np.random.choice(a, size=None, replace=True, p  
=None)
```

Sampling design

In a simple random sample, every sample has the same chance of being selected.

Types of randomness

1. Random
2. Uniform
3. Normal/Gauss
4. Triangular
5. Randint
6. Shuffle
7. Choice/Choices
8. Sample

No.	Function	Output
1	random()	Float: $0.0 \leq X < 1.0$
2	uniform(a, b)	Float: $a \leq X \leq b$
3	gauss(mu, sigma)	Float
4	triangular(a, b, m)	Float: $a \leq X \leq b$, with mode m
5	randint(a, b)	Int: $a \leq X \leq b$
6	shuffle(list/array/...)	None
7	choice/choices(list/array/..., k=...)	An item/list
8	sample(list/array/..., k=...)	A list of unique items

Case study: Fuel tank experiment

- Your company have just released a new fuel tank design for a rocket.
- People are interested in the durability of the product.
- Can be test this with our traditional experiment design?

Think about this

- Can you be 100% sure that your product is durable?
- If not, what will you do?

Preparation







Sample	123	235	246	357
P				



Sample	123	235	246	357
P	$\frac{2}{3}$	$\frac{1}{3}$	1	0



Sample	123	235	246	357
P	$2/3$	$1/3$	1	0



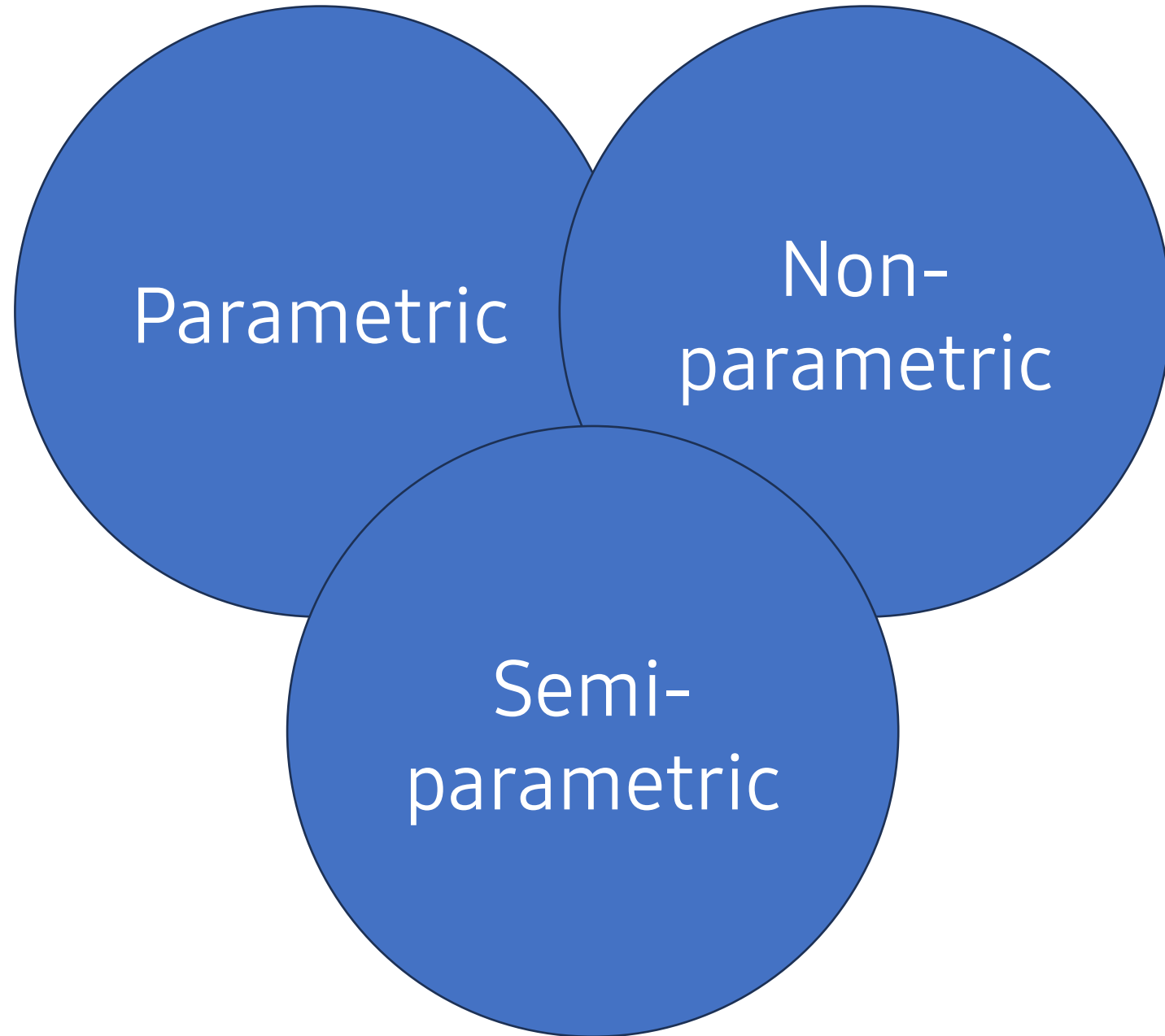
- Prop. of 3 selected tanks are all failed?



Proportion of failures	No. of samples	Fraction of samples
0	1	$1/35 \sim 0.03$
$1/3$		
$2/3$		
1	4	$4/35 \sim 0.11$
Total	35	1

Think bigger

- You can use this method to Find appropriate n .
 - Find n tanks for the rocket.
 - Find n records that meets the expectations.



One-run

```
urn = [1, 1, 0, 1, 0, 1, 0]  
sample = np.random.choice(urn, size=3, replace=False)  
print(f"Sample: {sample}")  
print(f"Prob Failures: {sample.mean()}")
```


Simulated experiment

```
urn = [1, 1, 0, 1, 0, 1, 0]
```

```
samples = [np.random.choice(urn, size=3, replace=False) for _ in  
range(1000)]
```

```
prop_failures = [s.mean() for s in samples]
```